PROJECT STUDY & PRESENTATION REPORT

CodeTutor

An interactive web application that explains any **Spring Boot + React** codebase — architecture, data flow, database diagram, code explanations and a question-answering assistant.

~8,600	~4,000	7	10
LINES OF JAVA	LINES REACT / CSS	INTERACTIVE TABS	AUTOMATED TESTS

Member A

20%

Platform, security, privacy & import pipeline

Member B

40%

Code-intelligence engine: parsing, graph, data flow

Member C

40%

Interactive product: UI, diagrams, search & chat

Technology: Spring Boot 3 · Spring Security + JWT · JPA · JavaParser · JGit · React 18 · Monaco Editor · React Flow · Vite

Contents

1. Team split at a glance — everyone

2. The project in one page — everyone

3. Requirements → what we built — everyone

4. System architecture — everyone

5. PART A — Platform, security & import pipeline — Member A · 20%

6. PART B — Code-intelligence engine — Member B · 40%

7. PART C — Interactive product: UI, diagrams, search & chat — Member C · 40%

8. Demo script (15 minutes) — all

9. Questions the examiners will ask — with answers — all

10. Honest limitations & future work — all

11. Appendix: REST API, commands, glossary — reference

> **How to use this report.** Read sections 1–4 together (everyone must know the big picture), then study **your own part** in depth (5, 6 or 7), rehearse the demo script in section 8, and practise the Q&A in section 9.

1. Team split at a glance

The split follows the architecture: **A** builds the safe foundation and the way code gets *into* the system, **B** builds the "brain" that understands the code, **C** builds everything the user *sees and asks*.

	Member A — 20%	Member B — 40%	Member C — 40%
Theme	Foundation: accounts, security, privacy, import	Brain: understanding Java/Spring/React code	Product: UI, diagrams, search, AI chat
Backend code	`auth/`, `security/`, `common/`, `project/`, `scanner/` (≈ 2,000 lines) + tests	`analysis/`, `flow/`, `explain/` (≈ 4,000 lines)	`search/`, `chat/` (≈ 1,100 lines)
Frontend code	`AuthPage`, `Dashboard`, `api.js`, `auth.jsx` (≈ 520 lines)	–	`Workspace` + all `views/`, `lib/`, `ui.jsx` (≈ 2,760 lines JS) + all CSS (≈ 700 lines)
Key requirements	User management, upload/validate, limits, privacy, .env consent, background processing	Tech detection, parsing, dependency graph, API/DB/module analysis, knowledge graph, data flow, explanations	Chunking, vector search, Q&A with source highlight, architecture/ER diagrams, read-only explorer, click-to-explain UI
Demo time	~3 min	~6 min	~6 min
Hardest ideas to explain	Why 404 not 403; zip-slip; async pipeline	Resolving method calls without a compiler; interface → implementation; Spring Data	TF-IDF retrieval; layered layout; why export used to crop

Project size: **≈ 8,600 lines of Java** (incl. the pre-existing login module), **≈ 3,300 lines of React/JS + ≈ 700 lines of CSS**, **10 automated tests**. Analysing the sample project *Spring PetClinic* (117 files) takes **~5 seconds**.

> **Rule for the presentation:** each person presents their own part, but *everyone* must be able to answer the general questions in §9.1.

---

2. The project in one page

Problem

Students struggle to understand large, unfamiliar codebases. They can download thousands of open-source projects, but reading files one by one is slow, and repositories rarely have beginner-friendly documentation.

Our solution — CodeTutor

A web app where a student **registers, uploads a GitHub link or a ZIP**, and gets an **interactive explanation of the project**:

What the student gets	Where
Private account; only they see their projects	Sign-in, dashboard
Tech stack with language percentages, key numbers, modules	*Overview* tab
The real folder structure, **read-only**, IDE-style colours; **click any file / class / method / field → short popup of what it does**	*Code* tab
Architecture diagram (layers: Front-end → Controllers → Services → Repositories → Entities → Database)	*Architecture* tab
Data-flow diagram for any endpoint (browser → security → controller → service → repository → table) and **how security works**	*Data flow* tab
Database / ER diagram (tables, PK/FK, 1:N relationships)	*Database* tab
API catalogue with access level (public / login / role)	*APIs* tab
Interactive dependency tree + libraries	*Dependencies* tab
Ask AI: questions answered from the code; relevant file opens with the lines highlighted; flow diagrams drawn in the chat	Chat panel
PNG export of every diagram	PNG button

How it works in one sentence

*We parse the code into an AST, extract classes / methods / calls / endpoints / tables / security rules, store them as a **knowledge graph** and as **semantic chunks in a vector index**, and every screen is just a different view of that data.*

Results we measured

Project	Files	Classes	Endpoints	Tables	Analysis time
Spring PetClinic	117	25	17	7	≈ 5 s
club_management (our own sample, Spring + React + JWT)	126	71	55	11	≈ 6 s
PNG export of the architecture diagram	–	–	–	–	≈ 1.3 s

3. Requirements → what we built

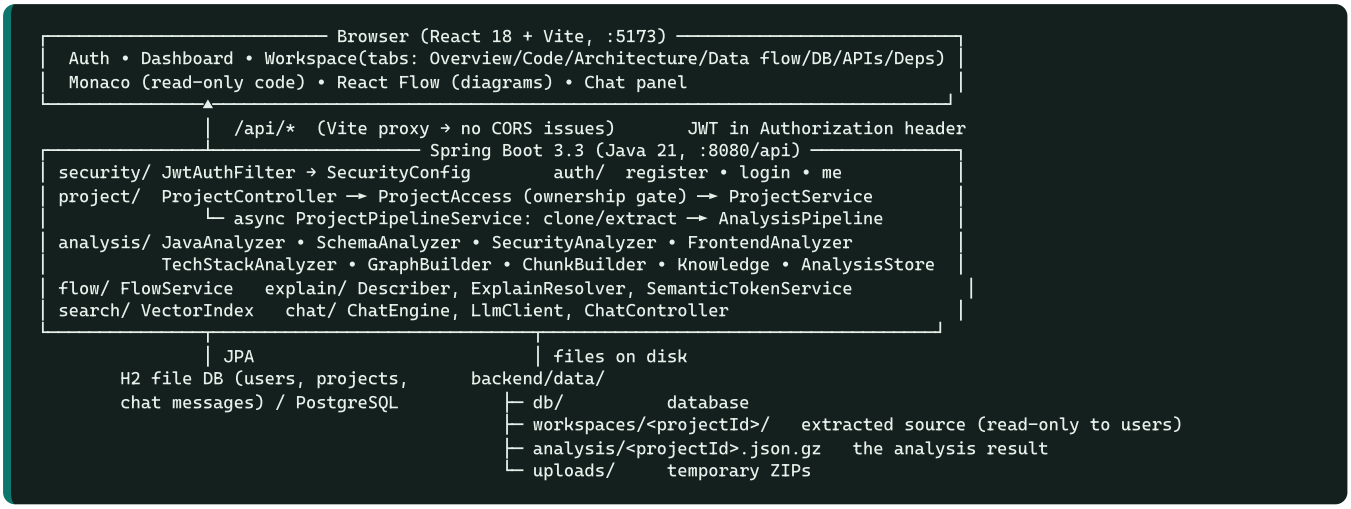
Mapping of the two requirement documents ("Requirements" and "System Design") to the implementation.

#	Requirement (document)	Status	Implemented in
R1	Upload GitHub link **or** ZIP	✅	`ProjectController`, `ProjectService`, `ProjectPipelineService`
R2	Architecture diagram	✅	`ArchitectureTab.jsx` (bands / detailed / modules)
R3	Clear schema (ER) diagram	✅	`SchemaAnalyzer` + `DatabaseTab.jsx`

#	Requirement (document)	Status	Implemented in
R4	Click file/class/method → short popup	✅	`Describer`, `ExplainResolver`, `SemanticTokenService`, `CodeViewer.jsx`, `ExplainCard.jsx`
R5	Browse files in same structure, **read-only**	✅	`ExplorerTab.jsx`, Monaco `readOnly`, `/file` endpoint
R6	Q&A using knowledge graph / vector DB, code opens & highlights	✅	`ChatEngine`, `VectorIndex`, `ChatPanel.jsx`
R7	Data-flow & security-flow explained, image generated	✅	`FlowService`, `FlowDiagram.jsx`, PNG export
F1	Register / login / logout	✅	`auth/`, `security/`, `AuthPage.jsx`
F2	Validate URL / file size / type	✅	`GitHubValidator`, `ZipValidator`, limits in `application.yml`
F3	Skip unneeded files; ask permission for `.env`	✅	`FileFilterService`, `AnalysisPipeline.isSensitive`, consent endpoint
F4	Technology detection + percentages	✅	`TechStackAnalyzer`
F5	Code parsing (AST)	✅ (JavaParser instead of Tree-sitter)	`JavaAnalyzer`
F6	Dependency analysis (graph)	✅ (own graph instead of NetworkX)	`GraphBuilder`, `Knowledge`
F7	API / DB / module detection	✅	`JavaAnalyzer.extractEndpoints`, `SchemaAnalyzer`, module assignment
F8	Knowledge graph	✅	`AnalysisModel.GraphNode / GraphEdge`
F9	Semantic chunks per module/layer/method	✅	`ChunkBuilder`
F10	Embeddings + vector search	⚠️ local TF-IDF vectors (no PGVector)	`VectorIndex`
F11	LLM explanation	⚠️ optional (key-based), default = graph-grounded	`LlmClient`, `ChatEngine`
F12	Chat history saved	✅	`ChatMessage`, `ChatController`
N1	Security / privacy	✅	per-user isolation, JWT, BCrypt, 404 on foreign data
N2	Resource limits	✅	`limits:` config
N3	Availability while analysing	✅	async pipeline + status polling
N4	Usability	✅	redesigned UI, light/dark theme
—	Redis / PGVector / LangGraph / Python workers	❌ replaced	see §9.4 for the honest explanation

4. System architecture (everyone reads this)

4.1 Big picture



4.2 Technology stack

Layer	Technology	Why
Frontend	React 18, Vite 5, React Router 7	fast dev server, component model
Code viewer	Monaco Editor 0.57 (bundled locally)	the editor of VS Code; read-only mode
Diagrams	`@xyflow/react` (React Flow 12) + `@dagrejs/dagre`	interactive nodes/edges, automatic layout
PNG export	`html-to-image`	DOM → image
Markdown in chat	`react-markdown` + `remark-gfm`	tables, lists in answers
Backend	Spring Boot 3.3.5, Spring Security, Spring Data JPA	as required by the design document
Auth	JWT (jjwt 0.12.6, HS512) + BCrypt(12)	stateless, scalable
Git	JGit 6.9 (shallow clone)	pure Java, no git binary needed
Code parsing	JavaParser 3.26 (Java 21 grammar)	accurate AST without compiling
Storage	H2 (file) by default, PostgreSQL profile	zero-setup dev, production option
Tests	JUnit 5, MockMvc, Spring Security Test	10 tests

4.3 End-to-end life of one project (memorise this — it connects all three parts)

```
1  User pastes GitHub URL / drops ZIP                                    [C → A]
2  Backend validates URL/ZIP, checks quota & project limit, saves Project(QUEUED)   [A]
3  Returns 202 immediately; @Async worker starts                          [A]
4  Worker clones (JGit depth=1) or safely extracts ZIP; deletes .git      [A]
5  AnalysisPipeline.analyze(...)                                          [B]
     scan files (skip junk, flag secrets) → tech stack → parse Java AST → classes/methods/calls
     → endpoints → security rules → DB schema → React files + API calls → modules
     → knowledge graph → explanations → semantic chunks
6  Result saved as gzip JSON; project becomes COMPLETED (stats copied to DB row)  [B + A]
7  UI polls /status every ~1.3 s; when ready it loads /overview           [C]
8  Every tab = a different read-only view of the same analysis:
```

```
      /files /file /semantic /explain  → Code tab                              [B data, C UI]
      /graph                           → Architecture tab                      [B data, C UI]
      /flow                            → Data-flow tab                         [B data, C UI]
      /schema                          → ER diagram                             [B data, C UI]
      /chat                            → vector search + graph → answer         [C]
```

4.4 The shared data model (`AnalysisModel.java`) — the "contract" between parts

All analysis output is plain Java `record`s, serialised to one JSON file per project:

```
public record Result(Overview overview, List<FileInfo> files, List<ClassInfo> classes,
                     List<EndpointInfo> endpoints, Schema schema, List<ModuleInfo> modules,
                     Security security, Frontend frontend, List<Library> libraries,
                     List<GraphNode> nodes, List<GraphEdge> edges, List<Chunk> chunks) {}
```

Record	Holds
`ClassInfo`	id (FQCN), name, package, file, line range, **kind** (class/interface/enum/record), **layer** (CONTROLLER, SERVICE …), **module**, annotations, fields, methods, `dependsOn` (+ kind), summary
`MethodInfo`	id, signature, params, return type, line range, annotations, `calls` (resolved targets)
`EndpointInfo`	HTTP method, path, full path, handler method id, params, return type, **access** (PUBLIC/AUTHENTICATED/ROLE:x)
`Schema`	`Table` (columns, PK/FK, nullable, type) + `Relation` (from→to, cardinality)
`Security`	enabled, mechanisms (JWT…), ordered **filter chain**, **auth rules**, session policy, password encoder
`GraphNode/GraphEdge`	the knowledge graph (CLASS, MODULE, TABLE, ENDPOINT, JSFILE; DEPENDS_ON, USES, EXTENDS, IMPLEMENTS, MAPS_TO, REFERENCES, HANDLED_BY, CALLS_API)
`Chunk`	`id` like `owner.ownercontroller.processfindform`, type (method/class/layer/module/endpoint/table/file), text, file + line range
`Flow` / `FlowStep`	computed on demand: a tree of steps (client → filters → controller → … → table)

`Knowledge` builds fast look-up maps on top of `Result` (callers, implementations, entity→table …) and is cached per project (LRU of 6).

4.5 Folder map

```
backend/src/main/java/com/codetutor/backend/
  auth/        User, AuthController, AuthService, JwtUtil …              (A)
  security/    SecurityConfig, JwtAuthFilter                             (A)
  common/      ApiResponse, GlobalExceptionHandler, NotFound/Limit/NotReady exceptions   (A)
  project/     Project entity, ProjectController, ProjectService, ProjectAccess,
               ProjectPipelineService, WorkspaceManagerService, AppLifecycle, validators (A)
  scanner/     FileFilterService (junk/secret filtering)                 (A)
  analysis/    AnalysisPipeline, JavaAnalyzer, SchemaAnalyzer, SecurityAnalyzer,
               FrontendAnalyzer, TechStackAnalyzer, GraphBuilder, Knowledge,
               AnalysisStore, ChunkBuilder, model/AnalysisModel          (B; ChunkBuilder → C)
  flow/        FlowService                                               (B)
  explain/     Describer, ExplainResolver, SemanticTokenService          (B)
  search/      VectorIndex                                               (C)
  chat/        ChatEngine, LlmClient, ChatController, ChatMessage        (C)
frontend/src/
  pages/       AuthPage, Dashboard (A) · Workspace (C)
  views/       OverviewTab … DependenciesTab, ChatPanel, CodeViewer, ExplainCard, diagram/shared  (C)
  lib/         layout, monaco, colors, format                            (C)
```

```
  api.js, auth.jsx (A) · ui.jsx, index.css, styles/ (C)
backend/src/test/   AnalyzerTest (B) · UserIsolationTest (A)
```

---

5. PART A — Platform, security & import pipeline (20%)

Owner: Member A **Slides:** 4–5 **Demo time:** ~3 min **You own:** accounts, JWT security, "*every user only sees their own projects*", safe import of GitHub/ZIP, limits, background processing, persistence, error handling, isolation tests.

5.1 What to remember (one paragraph)

> *"We built the secure foundation. Users register and log in; the server issues a signed JWT; every request passes a filter that identifies the user. Every project query is scoped by the logged-in user, so another user's project returns 404 — it is indistinguishable from a project that doesn't exist. Importing a repository never blocks the request: we validate, save the project as QUEUED, return immediately, and a background worker clones/extracts it with strict limits, then runs analysis."*

5.2 Authentication (JWT + BCrypt)

Flow: `POST /auth/register` → password hashed with **BCrypt strength 12** → user saved → JWT access token (24 h) + refresh token (7 d) returned. `POST /auth/login {identifier, password}` accepts **email or username**. The frontend stores the access token in `localStorage` and sends `Authorization: Bearer <token>` on every call (axios interceptor).

Why JWT? Stateless: no server session, so the app scales horizontally; the token itself carries the user (`sub` = email) and is signed (HS512) so it can't be forged.

`SecurityConfig` — the whole security policy in one place

```
http
    .csrf(AbstractHttpConfigurer::disable)                      // stateless → no CSRF cookie
    .cors(cors -> cors.configurationSource(corsConfigurationSource()))
    .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/me").authenticated()        // protected (/auth/** is public)
            .requestMatchers("/auth/**").permitAll()            // register / login
            .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
            .anyRequest().authenticated())                      // EVERYTHING else needs a valid token
    .exceptionHandling(ex -> ex.authenticationEntryPoint((req, res, e) -> {
        res.setStatus(401);                                     // clean JSON 401
        res.setContentType("application/json");
        res.getWriter().write("{\"success\":false,\"message\":\"Authentication
required\",\"error\":\"UNAUTHORIZED\"}");
    }))
    .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
```

> **Bug we fixed (good story):** originally `/auth/**` was fully public, so `GET /auth/me` without a token reached the controller with a `null` user and returned **500**. Now `/auth/me` is explicitly authenticated and unauthenticated calls return a proper **401**.

`JwtAuthFilter` runs before every request: reads the header, validates the token, loads the user, puts it in the `SecurityContext`. We made it robust to **stale tokens** (token of a user that no longer exists, e.g. after a database reset):

```
try {
    userDetails = userDetailsService.loadUserByUsername(username);
} catch (Exception e) {
    // token of a user that no longer exists → treat as anonymous → 401, not 500
    filterChain.doFilter(request, response);
    return;
}
```

On the frontend, `api.js` catches any 401 and fires an `auth:expired` event; the `AuthProvider` clears the token and the router sends the user to the login page.

5.3 Privacy: "each user only sees their own projects" (the headline requirement)

Three layers of defence:

① Database queries are always scoped by owner

```
// ProjectRepository
Optional<Project> findByIdAndOwnerEmail(Long id, String ownerEmail);
List<Project>     findByOwnerEmailOrderByCreatedAtDesc(String ownerEmail);
```

② One gate for every endpoint — `ProjectAccess`

```
public Project owned(String email, long projectId) {
    return projects.findByIdAndOwnerEmail(projectId, email)
            .orElseThrow(() -> new NotFoundException("Project not found"));
}

public Ctx ready(String email, long projectId) {          // owned + analysed
    Project p = owned(email, projectId);
    if (p.getStatus() != COMPLETED) throw new NotReadyException(...);
    Knowledge k = store.load(projectId).orElseThrow(...);
    return new Ctx(p, user(email), k, workspaces.getWorkspaceDir(projectId));
}
```

Every controller method starts with `access.ready(...)` or `access.owned(...)` — there is **no way to reach analysis data without passing this gate**.

③ **Why 404 and not 403?** A 403 ("forbidden") would *confirm that project 42 exists* and belongs to somebody else. Returning **404** for both "doesn't exist" and "belongs to someone else" leaks nothing.

Extra isolation details:

- Chat messages store `(projectId, userId)`; queries use both (`findByProjectIdAndUserIdOrderByIdAsc`).
- Files are served only through `/projects/{id}/file?path=`, which normalises the path and rejects anything outside the project's workspace (`..` traversal) and `.git`.
- Frontend: the whole authenticated app is rendered as `<div key={user.id}>` — switching account **remounts everything**, so no stale data from the previous user can appear.

The test that proves it (`UserIsolationTest`) — registers Alice and Bob, gives Alice a project, then asks as Bob:

```
for (String path : new String[]{"", "/status", "/overview", "/graph", "/endpoints", "/schema", "/classes",
"/files",
        "/file?path=pom.xml", "/semantic?path=A.java", "/explain?kind=file&id=pom.xml", "/symbols?q=user",
"/dependencies",
        "/chat/history", "/chat/suggestions", "/flow?endpoint=security"}) {
    mvc.perform(get(base + path).header("Authorization", "Bearer " +
bobToken)).andExpect(status().isNotFound());
```

```
}
// POST /chat, POST /reanalyze, DELETE → also 404; and the project still exists for Alice.
```

Run it: `cd backend && mvn test` → **5 isolation tests + 5 analyzer tests = 10 green**.

5.4 Import pipeline (clone / ZIP) — asynchronous and safe

Step 1 — request returns immediately (`ProjectService.createFromGithub`)

```
User owner = access.user(email);
checkQuota(owner, 0);                                        // max projects + storage quota (5 GB / user)
GitHubValidator.GitHubRepoInfo info = gitHubValidator.validateAndExtract(req.getGithubUrl());
Project p = Project.builder()....owner(owner).build();
p.setStatus(Project.AnalysisStatus.QUEUED);
p = projects.save(p);                                        // committed BEFORE the worker starts
pipeline.importGithub(p.getId(), info.cloneUrl());           // @Async → returns at once
return ProjectResponse.from(p);                              // HTTP 202 Accepted
```

> The create methods are **deliberately not** `@Transactional`: the row must be committed before another thread looks it up.

Step 2 — background worker (`ProjectPipelineService`), on a bounded pool (`core=2, max=3, queue=30`):

```
@Async("analysisExecutor")
public void importGithub(long projectId, String cloneUrl) {
    run(projectId, () -> {
        update(projectId, CLONING, 5, "Cloning repository from GitHub…");
        Path dir = workspaces.createOrCleanWorkspace(projectId);
        try (Git g = Git.cloneRepository().setURI(cloneUrl).setDirectory(dir.toFile())
                        .setDepth(1).setCloneAllBranches(false).setTimeout(180).call()) { }
        workspaces.deleteTree(dir.resolve(".git"));          // history isn't needed → saves space
    });
}
```

`run(...)` is the common wrapper: fetch → `pipeline.analyze(...)` → `store.save(...)` → `finish(...)`; it catches `LimitExceededException` (shows the message and cleans up) and any other exception (shows a **sanitised** message — server paths are stripped).

ZIP extraction — defences (`extractZip`)

```
try (ZipFile zf = openZip(zip)) {                            // central-directory based: handles
Windows/Mac/7-Zip archives
    for (ZipEntry entry : Collections.list(zf.entries())) {
        if (++entries > maxZipEntries) throw new LimitExceededException("The archive contains too many
entries…");
        String name = entry.getName().replace('\\', '/');    // Windows tools write backslashes
        if (ignoredZipPath(name)) continue;                  // node_modules, .git, __MACOSX, target,
dist, build…
        Path out = target.resolve(name).normalize();
        if (!out.startsWith(target)) throw new LimitExceededException("unsafe path");   // ZIP-SLIP
protection
        …copy bytes, counting…  if (total > maxRepoBytes) throw new LimitExceededException(…);  // ZIP-
BOMB protection
    }
}
flattenSingleRoot(dir);   // GitHub zips wrap everything in one folder → move it up so paths look like the
repo
```

Attack / problem	Defence
Zip-slip (`../../evil` entry)	`normalize()` + `startsWith(target)` check
Zip bomb (tiny zip → gigabytes)	count extracted bytes, abort at `max-repo-bytes`
Millions of entries	`max-zip-entries`
Huge upload	multipart 100 MB limit → HTTP 413 with friendly message
Fake `.zip`	magic-bytes check (`PK`) in `ZipValidator`
Giant repository	`max-files` , `max-repo-bytes` checked again when scanning
Server restarts mid-analysis	`AppLifecycle.onReady` marks stuck projects FAILED ("press Re-analyse") and deletes orphan workspace folders

Limits (`application.yml`)

```
limits:
  max-files: 8000
  max-repo-bytes: 209715200      # 200 MB of extracted source
  max-zip-entries: 20000
  max-file-bytes: 2097152        # files > 2 MB are not parsed
  max-projects-per-user: 10
```

5.5 Sensitive files (`.env` , keys) — "ask the user first"

`AnalysisPipeline.isSensitive(name)` flags `.env*` , `*.pem` , `*.key` , `id_rsa*` , `credentials.json` … They are **not read, not listed, not indexed**; the Overview shows "*1 sensitive file was skipped — Allow access*". Only after the user clicks, `POST /projects/{id}/sensitive-access` stores the allowed paths on the project and re-analyses. The `/file` endpoint also refuses to serve such a file unless allowed (checked directly in `ProjectController.file`).

5.6 Persistence

- **H2 file database** (`backend/data/db`) → users, projects, chat messages survive restarts. `ddl-auto: update` creates tables.
- **PostgreSQL** with `--spring.profiles.active=postgres` (`application-postgres.yml`).
- Analysis result → `data/analysis/<id>.json.gz` (gzip JSON), cached in memory (`AnalysisStore` , LRU 6).
- Per-user storage meter: `User.storageUsedBytes` is recomputed from the sum of the user's project sizes.

5.7 Error handling

`GlobalExceptionHandler` converts exceptions into one JSON shape `{success, message, error}` with sensible status codes:

Exception	HTTP	Meaning
`NotFoundException`	404	missing **or not yours**
`NotReadyException`	409	project still analysing / failed
`LimitExceededException` , `MaxUploadSizeExceededException`	413	size/entry limits

Exception	HTTP	Meaning
`BadCredentialsException`	401	wrong password
`IllegalArgumentException`, validation errors	400	bad input
anything else	500	logged, generic message to client

5.8 Your frontend (small): `AuthPage`, `Dashboard`

- **AuthPage**: split layout, Sign-in / Create-account tabs, inline errors from the API (`errMsg`), animated "request path" strip.
- **Dashboard**: import card (GitHub URL / drag-drop ZIP **with upload progress %**), project cards with live progress bar. It polls `GET /projects` every 1.8 s *only while something is analysing*:

```
const busy = projects?.some((p) => PROCESSING.has(p.status));
useEffect(() => {
  if (!busy) { refresh(); return undefined; }
  const t = setInterval(load, 1800);
  return () => clearInterval(t);
}, [busy, load, refresh]);
```

5.9 Demo for Part A (≈ 3 min)

1. Register a new user (show validation message for weak password / duplicate email).
2. Paste `https://github.com/spring-projects/spring-petclinic` → show **card with progress bar** (point out: "*the UI never froze*").
3. Open DevTools → Network: show `POST /api/projects/github` returned **202** instantly.
4. Log in as a second user in a private window → project list is empty; paste project URL of user 1 → "*Project not found*".
5. Show `UserIsolationTest` passing in the terminal.

5.10 Your likely questions → see §9.2

---

6. PART B — Code-intelligence engine (40%)

Owner: Member B **Slides:** 8–10 **Demo time:** ~6 min **You own:** everything that *understands* the code: `AnalysisPipeline`, `JavaAnalyzer`, `SchemaAnalyzer`, `SecurityAnalyzer`, `FrontendAnalyzer`, `TechStackAnalyzer`, `GraphBuilder`, `Knowledge`, `FlowService`, `Describer`, `ExplainResolver`, `SemanticTokenService`.

6.1 What to remember (one paragraph)

> *"We don't use AI to guess what the code does — we **parse** it. JavaParser gives us the AST of every file. From it we extract classes, methods and fields, detect each class's role from Spring annotations, **resolve which method calls which** (even through interfaces and Spring Data repositories) and build a knowledge graph. On top of the graph we generate endpoint lists, the database schema, security rules and data-flow diagrams. Because it's static analysis, the results are exact and reproducible."*

6.2 The pipeline (`AnalysisPipeline.analyze`)

#	Step	Class	Output
1	Scan files: skip junk (`node_modules`, `target`, `build`, `.git` …), flag secrets, enforce limits, count lines	`AnalysisPipeline.scan` + `FileFilterService`	`List<FileInfo>`
2	Tech stack: read `pom.xml`, `package.json`, Gradle, requirements, Docker/CI files; language %	`TechStackAnalyzer`	frameworks, DBs, tools, **context-path**
3	Parse Java: classes, methods, calls, dependencies, layers, modules, endpoints	`JavaAnalyzer`	`List<ClassInfo>`, `List<EndpointInfo>`
4	Security rules + per-endpoint access	`SecurityAnalyzer`	`Security`
5	Database schema	`SchemaAnalyzer`	`Schema`
6	React files, imports, API calls → linked to endpoints	`FrontendAnalyzer`	`Frontend`
7	Modules, graph, explanations (class/file summaries)	`GraphBuilder`, `Describer`	nodes/edges
8	Semantic chunks	`ChunkBuilder` (C explains it)	`List<Chunk>`

Progress is reported through a callback (`10%, 25%, 40% …`) that the import worker writes to the project row — that is what the UI progress bar shows.

6.3 `JavaAnalyzer` — the core (875 lines)

It works in **seven passes** over all Java files:

```
// 1. parse + collect declarations
for (String rel : javaFiles) { ParseResult<CompilationUnit> pr = parser.parse(p); collect(rel,
pr.getResult().get()); }
// 2. layers, fields
```

```
for (CB c : classes.values()) detectLayer(c);
for (CB c : classes.values()) resolveFields(c);
// 3. method calls
for (CB c : classes.values()) resolveCalls(c);
// 4. dependencies
for (CB c : classes.values()) computeDependencies(c);
// 5. modules          6. endpoints          7. materialise into immutable records
```

JavaParser is configured for **Java 21** (records, text blocks, switch expressions) and needs **no classpath / no compilation**.

(a) Layer detection — what role does a class play?

Mostly from Spring annotations, then from naming/superclasses:

```
if (c.test)                                        layer = "TEST";
else if (an.contains("SpringBootApplication"))     layer = "APPLICATION";
else if (an.contains("ControllerAdvice") || an.contains("RestControllerAdvice")) layer = "ADVICE";
else if (an.contains("RestController") || an.contains("Controller"))           layer = "CONTROLLER";
else if (an.contains("Entity") || an.contains("MappedSuperclass") || …)        layer = "ENTITY";
else if (an.contains("Service"))                   layer = "SERVICE";
else if (an.contains("Repository") || (c.isInterface && supers.stream().anyMatch(REPO_BASES::contains)))
layer = "REPOSITORY";
else if (supers.stream().anyMatch(FILTER_BASES::contains) || an.contains("EnableWebSecurity") …) layer =
"SECURITY";
…  // CONFIG, COMPONENT, EXCEPTION, DTO (by name suffix), UTIL, MODEL …
```

For Spring Data repositories it also remembers the **entity and id types** from `JpaRepository<Owner, Integer>` (needed later to map repository → table).

(b) Name resolution without a compiler (`JavaIndex.resolve`)

When code says `OwnerRepository`, which class is that? Order of rules (same as Java): **nested class** → **explicit import** → **same package** → **wildcard import** → **unique simple name in project**. Anything not found = external library (JDK/Spring) → ignored.

(c) Call resolution — "who calls whom" (the hardest part)

For each method call `x.foo(...)` we need the **type of** `x`. We compute it from a scope map (fields + parameters + local variables, `var` inferred from the initialiser):

```
private String typeOf(Expression e, CB ctx, Map<String,String> scope, int depth) {
    if (e instanceof NameExpr ne)      return scope.get(ne.getNameAsString());      // local / param /
field
    if (e instanceof ThisExpr)         return ctx.name;
    if (e instanceof ObjectCreationExpr oc) return oc.getType().asString();         // new Owner()
    if (e instanceof FieldAccessExpr fa …)  // this.owners  → field type
    if (e instanceof MethodCallExpr mc) {   // chained: ownerRepository.findById(id).orElseThrow()
        …return type of the called method (project classes), or…
        // Spring Data knowledge: findById → Optional<Entity>, save → Entity, findAll* → List<Entity>
        if (Set.of("findById","findOne").contains(name)) return "Optional<" + ent + ">";
        if (Set.of("save","saveAndFlush","getReferenceById").contains(name)) return ent;
    }
    …
}
```

- Calls on project classes are stored as `Call(targetClass, method, line, targetMethodId)`.
- Calls on a Spring Data repository method that doesn't exist in our source (e.g. `save`) are stored with `library=true` — the flow generator turns them into "*Spring Data generates INSERT/UPDATE*" steps.

- **Noise filter:** trivial getters/setters on entities/DTOs are skipped (`getId()` , `setName()`), otherwise every flow would be drowned in them.

(d) Dependencies between classes — with a *kind*

`computeDependencies` collects references from extends/implements, field types (incl. generics), constructor/ `@Bean` parameters, method signatures, `new X()` , static references and calls. Each target gets one **primary kind**, in this precedence: `EXTENDS > IMPLEMENTS > INJECTS > CALLS > USES` .

- **INJECTS** = field is `@Autowired` , or a constructor parameter type in a Spring bean, or `final` field with Lombok constructor annotation.

(e) Modules — the "User / Product / Order" idea of the design document

```
String base = commonPrefix(pkgs);                       // e.g. com.college.clubmanagement
boolean byLayer = ≥ 70% of first package segments ∈ {controller, service, repository, dto, …}
module = byLayer ? stem(c)           // UserController, UserService, User  → "user"
                 : firstSegment(pkg) // com.shop.order.* → "order"
foldMinorModules();   // DTO/enum/exception-only names fold into the nearest real module or "common"
```

Result on our sample: 39 raw "modules" → **22 meaningful** ones after folding.

(f) Endpoint extraction

Combines class-level `@RequestMapping` prefix + method-level `@GetMapping/@PostMapping/…` , supports arrays and `method = RequestMethod.X` , extracts parameters (`@PathVariable` , `@RequestParam` , `@RequestBody`), detects whether a `@Controller` returns a **view** or JSON. `fullPath` prepends the `server.servlet.context-path` read from `application.yml` .

6.4 `SchemaAnalyzer` — database diagram without a database

Input: JPA entities (annotations) **and** `CREATE TABLE` scripts. **Output:** tables, columns (type, nullable, unique, PK, generated, length), FKs, relations with cardinality.

How JPA annotations become schema:

Annotation	Result
`@Entity` , `@Table(name="owners")`	table (name default = snake_case of class)
`@Id` , `@GeneratedValue`	primary key, auto-generated
`@Column(nullable=false, length=30)` , `@NotBlank` , `@Size(max=)`	NOT NULL, length
`@ManyToOne @JoinColumn(name="owner_id")`	FK column on this table + relation **N:1**
`@OneToMany(mappedBy=…)`	no column (the other side owns it)
`@OneToMany @JoinColumn` (unidirectional)	FK column is added to the **child** table
`@ManyToMany @JoinTable(...)`	**join table** with two FK columns + two relations
`@MappedSuperclass` parent (e.g. `BaseEntity`)	its fields are inherited into every child table
`@Embedded`	embeddable's columns flattened into the table

```
// ManyToMany → join table
String joinTable = jt != null && jt.arg("name") != null ? unq(jt.arg("name")) : table + "_" + targetTable;
jcols.add(new Column(jc, sqlFor(entity, pk1), …, true /*PK*/, …, table + "." + pk1, …));   // FK → owner
```

```
side
jcols.add(new Column(ic, sqlFor(target, pk2), …, true, …, targetTable + "." + pk2, …));    // FK → other
side
relations.add(new Relation(joinTable, table, jc, pk1, "MANY_TO_ONE", "N:1", …));
relations.add(new Relation(joinTable, targetTable, ic, pk2, "MANY_TO_ONE", "N:1", …));
```

SQL scripts are parsed with regex (`CREATE TABLE … ( … )` , `PRIMARY KEY` , `FOREIGN KEY … REFERENCES`); when several dialect files exist (H2/MySQL/Postgres) the **H2** one is preferred; SQL info **enriches** the JPA columns (exact types, constraints). *Result for PetClinic:* `owners, pets, visits, types, specialties, vets, vet_specialties` + 5 relationships (including the join table).

6.5 `SecurityAnalyzer` — "how is security implemented?"

It re-parses the security config and walks the method-call chain of `HttpSecurity` :

```
.authorizeHttpRequests(a -> a
     .requestMatchers("/auth/**").permitAll()
     .requestMatchers(HttpMethod.GET,"/api/products/**").permitAll()
     .requestMatchers("/api/admin/**").hasRole("ADMIN")
     .anyRequest().authenticated())
```

Each `permitAll / authenticated / hasRole …` call is paired with the matcher call in its scope → **ordered rule list** `(pattern, method, access)` . It also detects: session policy (`STATELESS`), CSRF disabled, CORS, `addFilterBefore(jwtFilter, …)` , password encoder (BCrypt…), `UserDetailsService` , token-service class, login endpoint (POST …/login).

Per-endpoint access is then evaluated exactly like Spring does (first matching rule wins), using Spring's own `AntPathMatcher` :

```
public static String accessFor(Security sec, EndpointInfo ep) {
    if (sec == null || !sec.enabled()) return "PUBLIC";
    for (AuthRule r : sec.rules()) {
        if (r.httpMethod() != null && !r.httpMethod().equalsIgnoreCase(ep.httpMethod())) continue;
        for (String p : r.pattern().split(" , "))
            if (p.trim().equals("/**") || MATCHER.match(p.trim(), ep.path())) return r.access();
    }
    return "AUTHENTICATED";
}
```

> **Bug story:** rules came out in the wrong order because JavaParser says a *chained* call "begins" where the whole chain begins (the same position for all rules). Fix: sort by the position of the **method name** (`mc.getName().getBegin()`). A unit test (`security_rules_are_ordered_and_evaluated`) now protects it.

6.6 `FrontendAnalyzer` + `TechStackAnalyzer`

- **React side:** regex finds components (`function Name(` , `const Name = () =>` , classes), relative imports (resolved to real files), and HTTP calls: `api.get('/x')` , `axios.post(...)` , `fetch(...)` . Template literals are normalised (`${id}` → `{var}`), the `/api` prefix/context path is stripped, then matched to a backend endpoint using Ant patterns → this is how a **flow starts in the browser** ("React UI → POST /auth/login").
- **Tech stack:** reads build files; language % is **lines of code**, but markup/config files are **capped at 400 lines each** so a generated 6,000-line CSS file can't make "CSS" the main language; `primaryLanguage` only considers real programming languages.

6.7 Knowledge graph (`GraphBuilder`, `Knowledge`)

Nodes: `MODULE`, `CLASS` (layer), `TABLE`, `ENDPOINT`, `JSFILE`. Edges: `CONTAINS`, `DEPENDS_ON` (injects/calls), `USES`, `EXTENDS`, `IMPLEMENTS`, `MAPS_TO` (entity→table), `REFERENCES` (FK), `HANDLED_BY` (endpoint→controller), `CALLS_API` (react file→endpoint), `IMPORTS`. `Knowledge` adds fast indexes: `callers`, `implementations` (interface → implementing classes), `entityTable`, `endpointsByHandler`, `dependents` …

6.8 Data-flow generator (`FlowService`) — requirement R7

`endpointFlow(id)` builds an ordered **tree of steps**:

1. **Client** — the React file that calls the endpoint (if found) else "browser/API client".
2. **Security chain** — each filter, then an **Authorization check** step showing the matched rule ("`/auth/**` → PUBLIC").
3. **Controller method** — plus any `@ModelAttribute` methods of that controller, which Spring runs *before* the handler ("runs first").
4. **Recursive call walk** over resolved calls (depth ≤ 7, ≤ 48 steps, cycle-safe), expanding **interface → implementation**, adding **Spring Data** steps and **TABLE** steps (read/write) from the repository's entity.
5. **Response** — HTML view or JSON.

```
for (MethodInfo tm : k.concreteTargets(target)) {          // interface method → its implementation(s)
    if (path.contains(tm.id())) continue;                  // avoid cycles
    if (!siblings.add(tm.id())) continue;                  // don't repeat the same call twice
    ClassInfo owner = k.classOf(tm);
    if (owner == null || skipTrivial(owner, tm)) continue; // skip DTO plumbing / one-line helpers
    int idx = add(steps, parent, owner.layer(), owner.name()+"."+tm.name()+"()", …, d.methodSummary(owner,
tm), …);
    if ("REPOSITORY".equals(owner.layer())) addTable(steps, idx, owner, tm.name());
    path.add(tm.id());  walk(steps, tm, idx, depth + 1, path);  path.remove(tm.id());
}
```

Real output for PetClinic `POST /owners/{ownerId}/edit`:

```
1. Client  — sends POST /owners/{ownerId}/edit
2. OwnerController.processUpdateOwnerForm()  — Spring routes the request here
3.    └ OwnerController.findOwner()   (@ModelAttribute — runs first)
4.         └ OwnerRepository.findById()  → reads table `owners`
5.    └ OwnerRepository.save()  — Spring Data generates INSERT or UPDATE on table `owners`   → writes table `owners`
6. HTML page — view engine renders the response
```

`securityFlow()` draws the filter chain (CORS → JWT filter → exception translation → authorization) with the rule list, ending in *allowed → controller* or *denied → 401/403*.

6.9 Explanations (`Describer`) — "short popup of what it does" (requirement R4)

No AI needed: descriptions are built from **real facts**:

- **Class:** layer + stem + injected dependencies + endpoints/tables, or the **Javadoc first sentence** if present. e.g. "*Web controller for owner: receives HTTP requests (13 endpoints, e.g. GET /owners/new …). It relies on OwnerRepository.*"
- **Method:** Javadoc, else a **verb phrase** from the name (`findByLastName` → "*Retrieves the owner by last name*"), plus "*It reads table* `owners`" (derived from repository calls), annotations (`@Transactional` → "Runs inside a database transaction").

```
VERBS.put("find","Retrieves"); VERBS.put("save","Creates / saves"); VERBS.put("delete","Deletes");
```

```
VERBS.put("login","Authenticates"); VERBS.put("validate","Validates") …
```

- **File:** pom.xml → "Maven build file: 31 dependencies and Java 17…", application.yml → "Spring Boot configuration: settings for server, spring, jwt…", SQL → "creates 7 tables", README → first paragraph, React file → component names + API calls. `ExplainResolver.at(file, line, word)` resolves a click position to the right symbol (call target → class name → method of this class → field → parameter).

6.10 `SemanticTokenService` — IDE-style colours

Re-parses the opened Java file and classifies **every identifier** into 14 kinds: `class, classDecl, interfaceDecl, method, methodDecl, field, fieldDecl, param, paramDecl, local, localDecl, annotation, constant, enumMember`. It uses scope information (is `owner` a parameter, a local, or a field?) and returns compact tuples `[line, column, length, kind, refIndex]`, where `refIndex` points to a resolved class/method id so a **click knows exactly what was clicked**.

6.11 Testing your part

`AnalyzerTest` writes a mini Spring project to a temp folder and asserts: layers & modules, INJECTS dependencies, resolved calls (`findById` on the repository), endpoints, schema FK `user_id → users.id`, NOT NULL/length, **security rule order and per-endpoint access**.

6.12 Demo for Part B (≈ 6 min)

1. *Overview tab*: stats, language %, layers, security summary, modules — "*all computed from the AST*".
2. *Code tab*: open `OwnerController.java` → colours (class/method/param/local different) → **click** `findPaginatedForOwnersLastName` → popup (what it does, calls, called by).
3. *Data flow*: select `POST /owners/{id}/edit` → explain each coloured step; tick "show each security filter".
4. *Data flow* → "*How security works*" on the JWT sample project (club_management): filters, rules, public vs login-required.
5. *Database tab*: ER diagram → click `pets` → columns, FKs, entity link.
6. *APIs tab*: access chips (ADMIN / Public) come from the security rules.

6.13 Your likely questions → see §9.3

---

7. PART C — Interactive product: UI, diagrams, search & chat (40%)

Owner: Member C **Slides:** 8–10 **Demo time:** ~6 min **You own:** `ChunkBuilder`, `VectorIndex`, `ChatEngine`, `LlmClient`, `ChatController`, and the whole React workspace (`Workspace`, all `views/`, `lib/`, styles).

7.1 What to remember (one paragraph)

> *"The workspace is one page with seven tabs; each tab is a different view of the same analysis. Code opens in Monaco in read-only mode with semantic colours; diagrams are drawn with React Flow with our own layout so they stay readable; and the chat answers by combining two 'brains' — the knowledge graph for structure and a vector index for meaning — so every answer comes with the exact code lines, which open highlighted."*

7.2 The two "brains" and chunking (design doc §13–15)

Semantic chunks (`ChunkBuilder`) — one chunk per meaningful unit, with ids exactly like the design document (`user.usercontroller`):

Chunk type	Example id	Content
method	`owner.ownercontroller.processfindform`	header (module, layer, file:line, summary) + code of the method (≤ 90 lines)
class	`owner.ownercontroller`	annotations, extends/implements, fields, method signatures
layer	`owner.controller`	all controllers of module *owner* with summaries
module	`owner`	module description + layer listing
endpoint / table / security	`endpoint.ownercontroller.processfindform`, `table.owners`, `security.overview`	facts in text form
file	`file.src/main/resources/application.yml.1`	60-line windows of config / SQL / README / React code

Vector index (`VectorIndex`) — a local TF-IDF embedding (no external service):

```
// tokens: split identifiers  findByLastName → [find, by, last, name], drop stop-words, light stemming
for (String t : tokens(header)) tf.merge(t, 3, Integer::sum);   // name/module/layer/file weighted ×3
for (String t : tokens(c.text())) tf.merge(t, 1, Integer::sum);
idf = ln((N + 1) / (df + 1)) + 1;                                  // rare words matter more
weight = (1 + ln(tf)) * idf;  → L2-normalised sparse vector

score = cosine(queryVector, chunkVector)
      + 0.25 if every word of the chunk's NAME appears in the question          // "OwnerController"
      + 0.08 per name word matched, +0.05 per name word matched via a synonym   // validation ~ validator
```

Query words are expanded with a synonym table (`login ⇔ auth/authenticate/jwt/token`, `database ⇔ table/entity/repository`, `delete ⇔ remove`, `create ⇔ add/save`…) at weight 0.35.

> **Honest statement for examiners:** the design document names *OpenAI embeddings / PGVector*. We implemented the same *idea* (embed chunks → nearest-neighbour search) with TF-IDF vectors so the app works offline with zero setup. The index sits behind one class, so a neural model or pgvector can replace it.

7.3 Chat engine (`ChatEngine`) — how a question becomes an answer

```
question → intent detection → entity linking → build "Draft" (markdown + sources + flow + actions)
                                          └→ (optional) LLM rewrites the Draft in natural language
```

Intent routing (first match wins) — simplified:

```
if ("GET /path" present and matches an endpoint)            return endpointFlow(eps);   // explicit
if (security words && (flow words || login/jwt/token…))      return security(ql);
if (schema words)                                            return schema(ql);        // table details /
ERD summary
if (endpoints matched && (flow words || "what happens"…))    return endpointFlow(eps); // draws a diagram
if (endpoint words + list words)                             return endpointList(...);
if (overview words)                                          return overview();
if (dependency words && class linked)                        return dependencies(cls);
if (method linked)                                           return explainMethod(...);
if (class linked strongly)                                   return explainClass(...);
else                                                         return search(q);         // vector search →
"Most relevant code"
```

Entity linking rules that made answers accurate:

- Endpoint match uses **coverage scoring** — each question word counts *once* by its best location (handler name 1.2 > path 1.0 > controller 0.3; synonyms 0.8/0.7), and ties prefer richer flows. (Before this, the word "owner" was counted three times and "owner search" opened the wrong endpoint.)
- A single-word class name (`Pet`) is a **strong** match only if written like code (`Pet`), not in plain text ("pet validation") → "Where is the pet validation implemented?" now correctly returns `PetValidator`.

Every answer carries

- `sources`: `[{file, startLine, endLine}]` → the UI opens the first one **highlighted** (requirement R6),
- `flow`: a `Flow` object → drawn inline as a diagram (requirement R7),
- `actions`: buttons such as "Open interactive flow", "Open ER diagram". Conversations are saved in `chat_messages (projectId, userId, role, content, sourcesJson, flowJson)`.

Optional LLM (`LlmClient`) — Anthropic / OpenAI / Gemini via plain `java.net.http`. If `LLM_API_KEY` is set, the *verified draft + top code chunks* are sent with a system prompt: *"Answer ONLY from the verified analysis and code excerpts … never invent classes."* Without a key everything still works. **No key is needed to run or demo the project.**

7.4 Frontend architecture

- **Routing:** `/login`, `/` (dashboard), `/project/:id/*` → `Workspace` with nested routes `overview | explorer | architecture | flow | database | apis | dependencies`.

- **State sharing:** `WorkspaceContext` gives every tab `project`, `overview`, and helpers `go(tab, query)`, `openCode(file,start,end)`, `ask(question, selection)`, `follow(link)`. Deep links work because state lives in the URL: `/project/1/explorer?file=…&start=94&end=96`.
- **Processing → Ready:** `Workspace` polls `/status` while `QUEUED/CLONING/ANALYZING`, shows a single progress card, then loads `/overview`.
- **API layer (`api.js`):** one axios instance, token interceptor, 401 → `auth:expired`; `errMsg()` turns any failure into a human sentence.
- **Design system:** CSS variables for light/dark (warm paper + deep-teal accent), one palette per architecture layer (`lib/colors.js`), motion helpers (`.stagger`, `rise`, count-up numbers, animated edges) that **respect** `prefers-reduced-motion`.

7.5 Code explorer (`ExplorerTab`, `CodeViewer`)

- **Monaco bundled locally** (no CDN): `lib/monaco.js` registers only needed languages (java, js, ts, xml, yaml, sql, markdown…) and a custom dark/light theme.
- **Read-only:** `options={{ readOnly: true, domReadOnly: true }}`; the file API only serves files from the project's own workspace.
- **Semantic colouring:** tokens from `/semantic` become Monaco decorations; the colours come from CSS variables so they follow the theme:

```
const decos = tokens.tokens.map(([line, col, len, kind, ref]) => ({
  range: new monaco.Range(line, col, line, col + len),
  options: { inlineClassName: `tok-${tokens.kinds[kind]}${ref >= 0 ? ' tok-clickable' : ''}` },
}));
decoRef.current = ed.createDecorationsCollection(decos);
```

- **Click → explain:** `onMouseDown` → find the token under the cursor → `GET /explain?ref=<resolved id>` (or file/line/word fallback) → `ExplainCard` popup, positioned so it never leaves the editor.
- **Highlight from chat:** `?start=&end=` → `createDecorationsCollection` with `isWholeLine` + `revealLinesInCenterIfOutsideViewport`.
- **Select code → "Ask AI about this"** floating button → sends the selection (file, lines, text) with the question.

7.6 Diagrams

Shared tooling (`diagram/shared.jsx`, `lib/layout.js`) — custom node types: class box, module, flow step, ER table, architecture band.

① **Architecture — Overview (default).** Bands stacked in request order; coloured header (layer colour, icon, description, count) + class names as pills; labelled arrows (`calls · 8`, `queries · 22`) with counts computed from graph edges; supporting layers (Security, Config, DTO…) in a side column.

```
// how many connections link two layers
graph.edges.forEach((e) => {
  if (['DEPENDS_ON','USES','MAPS_TO'].includes(e.type)) bump(layerOf.get(e.source),
layerOf.get(e.target));
  else if (e.type === 'CALLS_API') bump('FRONTEND', layerOf.get(handler.get(e.target)));   // React file →
controller
});
```

Band heights are *estimated* from the wrapped pill rows so bands never overlap; the view opens fitted to **width** (readable) and scrolls vertically. **Detailed** = every class on swim-lanes; **Modules** = one box per module with aggregated links.

② **Swim-lane layout algorithm (`layeredLayout`)** — for the Detailed view: nodes are grouped by layer (rows); inside each row the order is improved with **barycentre sweeps** (8 passes up/down: put each node at the average position of its neighbours in the adjacent layers) → fewer edge crossings. Rows with many nodes wrap.

③ **Data flow (`FlowDiagram`)** — `dagre` top-to-bottom layout; the run of security filters is **collapsed into one node** ("Security filters 2–4") with a checkbox to expand; edges animated (dash flow) to show direction; click a node → highlights its edges and shows the description.

④ **ER diagram (`DatabaseTab`)** — `dagre` left-to-right; `TableNode` shows PK/FK badges and types; custom `RelationEdge` draws **N** at the FK side and **1** at the referenced side with `EdgeLabelRenderer`.

⑤ **Dependency tree** — lazy: roots are controllers; expanding calls `/dependencies?class=…&dir=out|in`; "Used by" direction answers "*what breaks if I change this class?*"; circular references are marked ↺.

7.7 PNG export — design & the bug we fixed (great story for the presentation)

Requirement: download the diagram as an image. Implementation (`DiagramTools`):

```
const b = renderedBounds(root, rf, nodes);      // size of EVERYTHING drawn (nodes + edges + labels)
measured from the DOM
const pad = 90;  const width = Math.ceil(b.width + pad*2), height = Math.ceil(b.height + pad*2);
markerClone = markers.cloneNode(true);  viewport.appendChild(markerClone);      // arrow heads live
outside the exported layer
const url = await toPng(viewport, {
  width, height, pixelRatio: Math.min(1.5, 5000 / Math.max(width, height)),
  skipFonts: true, fontEmbedCSS: '',            // ← do not fetch/embed web fonts: this made export slow
  style: { width, height, transform: `translate(${pad - b.x}px, ${pad - b.y}px) scale(1)`,
transformOrigin: '0 0' },
});
```

Problem reported	Root cause	Fix
Image cropped (right column & bottom row missing)	bounds computed from node *data* — missed unmeasured nodes and edge labels	measure the **real rendered DOM rectangles** of nodes, edges and labels
Arrow heads missing in PNG	SVG `<marker>` definitions are outside the exported element	clone the marker `<svg>` into the viewport during capture
Slow download	`html-to-image` fetched and base64-embedded Google web fonts every time	`skipFonts: true` + system font stack in the export; lower pixel ratio. Export now ≈ **1.3 s**

7.8 Other UI details worth mentioning

- **Command palette** (Ctrl+K) → `/symbols?q=` searches classes, methods, endpoints, files.
- **Overview tab:** count-up statistics, language bar, layer bars, security card, "Allow access" banner for sensitive files, suggested questions from the backend.
- **APIs tab:** fixed grid columns with ellipsis (fixed an overlapping-text bug), access chips, expandable rows with parameters.
- **Theme:** light/dark toggle persisted in `localStorage`; React Flow gets `colorMode`; Monaco theme switches live.

7.9 Demo for Part C (≈ 6 min)

1. *Architecture* → Overview bands, hover/click a class → popup; switch to *Detailed* and *Modules*; press **PNG** → open the image (show it is complete).
2. *Data flow* → toggle "show each security filter"; click a step → "Show code" opens the file at that line.
3. *Ask AI* → ask **"How does the owner search work?"** → diagram appears in the chat, code opens highlighted.
4. Ask **"Where is the pet validation implemented?"** → `PetValidator` (vector search).
5. Select a few lines in the editor → **Ask AI about this** → explanation.
6. Toggle dark theme.

7.10 Your likely questions → see §9.4

---

8. Demo script (15 minutes)

Before you start (checklist):

- ☐ `start-dev.cmd` (or two terminals) — wait until the backend prints `Started CodeTutorApplication`.
- ☐ Open http://localhost:5173 — **pre-analyse** two projects beforehand (PetClinic from GitHub + our Spring/React/JWT project) because cloning needs internet.
- ☐ Keep a ZIP of a small project ready as a **backup** in case GitHub is slow.
- ☐ Browser zoom 100 %, one normal window + one private window (for the second user).
- ☐ Optional: set `LLM_PROVIDER` / `LLM_API_KEY` — **not required**.

Time	Who	What to show	Key sentence
0:00–1:00	all	Problem + solution (slide §2)	"Students can't read big codebases quickly."
1:00–4:00	**A**	Register → import PetClinic → progress → privacy (2nd user can't see it) → isolation test	"Foundation first: secure, private, and never blocks."
4:00–6:30	**B**	Overview → click class/method in code → data flow of an endpoint	"We parse, we don't guess."
6:30–9:30	**B**	Security flow (JWT project) → ER diagram → APIs with access levels	"Rules, tables and access levels are extracted automatically."
9:30–12:30	**C**	Architecture overview/detailed/PNG → chat questions with highlighted code	"Two brains: graph for structure, vectors for meaning."
12:30–14:00	**C**	Select code → Ask AI → dark theme → Ctrl+K	"Everything is one click away."
14:00–15:00	all	Limitations + future work (§10) → questions	"Honest about what is simplified."

9. Questions the examiners will ask — with answers

9.1 General (everyone should answer these)

1. **What problem does it solve?** Students can't understand big unfamiliar codebases quickly; we generate architecture, data flow, DB diagram, explanations and Q&A automatically.
2. **Is it AI?** The *understanding* is deterministic static analysis (AST parsing + graph). AI is optional and only rewrites verified facts into natural language. This avoids hallucination — every statement is traceable to code lines.
3. **What languages are supported?** Deep analysis: **Java Spring Boot + React** (as the document says). Other files are scanned, listed and viewable.
4. **How big a project can it handle?** Limits: 8,000 files / 200 MB source / 2 MB per parsed file / 100 MB upload (configurable). PetClinic ≈ 5 s.
5. **What would you build next?** See §10.

9.2 Part A questions

- **Why JWT and not sessions?** Stateless → scales horizontally, simple SPA integration; token is signed (HS512) so it can't be altered. Trade-off: can't be revoked before expiry (we'd add a blacklist/refresh rotation).
- **Where is the token stored? Is that safe?** `localStorage`. Simple; vulnerable to XSS. Production alternative: httpOnly cookie + CSRF protection. (We disable CSRF because we don't use cookies.)
- **How do you stop one user seeing another's project?** Owner-scoped queries + the single `ProjectAccess` gate + 404 responses + tests that try every endpoint as another user.
- **Why 404 instead of 403?** 403 would reveal the project exists. 404 leaks nothing.
- **What if someone uploads a malicious ZIP?** Zip-slip (path check), zip-bomb (byte limit), entry limit, magic-bytes check, skipped dirs; extraction happens inside an isolated per-project folder; files are never executed.
- **Why is the import asynchronous?** Cloning/analysis can take seconds to minutes; HTTP request would time out and block the UI. We return 202 and poll progress.
- **What happens if the server restarts during analysis?** On startup `AppLifecycle` marks such projects FAILED with a "Re-analyse" hint and removes orphan folders.
- **Where is redis?** The document proposes Redis for background jobs; we use an in-process bounded thread pool (same behaviour for one server). With multiple servers we'd move the queue to Redis.
- **How are passwords stored?** BCrypt (strength 12), never plain text.
- **What about `.env` files?** Never read unless the owner explicitly allows it; the file API blocks them too.

9.3 Part B questions

- **Why JavaParser instead of Tree-sitter?** The design doc suggests Tree-sitter (+ Python). We're a Java backend, and JavaParser gives a typed Java AST directly in-process — no native bindings or second runtime. Same concept: source → syntax tree.
- **How do you know which method is called without compiling?** We track the declared type of each variable (fields, parameters, locals) and resolve method names against our own classes; interfaces map to

implementations; Spring Data methods use `JpaRepository<Entity,Id>` knowledge. Unresolvable calls (JDK/third-party) are simply ignored.

- **What are the limits of that?** Reflection, lambdas with inferred types, generics-heavy code, dynamic proxies. Those calls may be missing from flows — it is *best effort*, not a compiler.
- **How do you detect layers?** Spring stereotype annotations first (`@RestController` , `@Service` , `@Repository` , `@Entity` …), then superclasses (`JpaRepository` , `OncePerRequestFilter`), then names/packages.
- **How do you build the ER diagram without a DB connection?** From JPA annotations (`@Entity/@Table/@Column/@ManyToOne/@JoinColumn/@JoinTable`) and `CREATE TABLE` scripts; the two are merged.
- **How do you find which endpoints are public?** We parse the `HttpSecurity` rule chain, keep rule order, and evaluate each endpoint against it with Spring's `AntPathMatcher` — first match wins, exactly like Spring.
- **How is a data-flow generated?** Start at the controller method and walk the resolved call graph depth-first (cycle-safe, depth ≤ 7), insert security filter steps before it and table steps after repositories.
- **Why are some steps skipped?** Getters/setters/DTO builders add noise; `skipTrivial` removes them.
- **How are explanations generated without AI?** From facts: layer, annotations, injected dependencies, Javadoc, naming (verb phrases), repository calls (reads/writes table). Deterministic, so the same code always gives the same text.
- **Does it work for Maven *and* Gradle?** Tech detection reads both (and npm/pip); dependency lists shown in the Dependencies tab.

9.4 Part C questions

- **How does the chat find the right code?** Two sources: the knowledge graph (exact names → classes/endpoints/tables/security) and a TF-IDF vector index over semantic chunks (meaning). Their results are merged into an answer with line references.
- **Why TF-IDF and not real embeddings / PGVector?** Needs no external service or API key and runs offline; code retrieval works well because identifiers are split into words and names are boosted. It's behind a single class so neural embeddings / pgvector can replace it.
- **How do you prevent hallucination?** The default answer is composed from analysed facts. Even with an LLM, the prompt only contains verified facts + retrieved code and forbids inventing classes.
- **Where does the "code opens and is highlighted" come from?** Each answer returns `sources` with file + line range; the chat navigates to `/explorer?file=…&start=…&end=…` ; Monaco decorates those lines and scrolls to them.
- **How is the code made read-only?** Monaco `readOnly: true` ; the backend exposes only `GET` for files — there is no write endpoint at all.
- **How do you colour different token types?** Backend (`SemanticTokenService`) classifies identifiers via the AST and scope; frontend applies them as decorations with CSS variables.
- **Why React Flow + dagre?** Interactive nodes/edges (click, zoom, pan) with automatic hierarchical layout; we added our own swim-lane layout for the detailed architecture view.
- **How did you make the architecture diagram understandable?** Two levels: a high-level band view (layers, counts, arrows) and a detailed class view; large graphs are never shown first.
- **Why did the PNG get cropped before, and how did you fix it?** (§7.7) bounds were computed from node data instead of rendered geometry; also arrow heads and fonts. We measure the DOM, copy the marker defs, and skip font embedding (1.3 s).

- **How do you handle theming?** CSS variables for colours; React Flow `colorMode`; Monaco theme switched on a custom event.
- **Is the UI responsive / accessible?** Layout adapts down to tablet widths; animations are disabled for users with `prefers-reduced-motion`; keyboard shortcut Ctrl+K, focus styles on inputs.

---

10. Honest limitations & future work

Limitation (say it before they ask)	Why	How we would fix it
Deep analysis only for **Spring Boot + React**	scope of the project statement	add analyzers per language (Python/Django via tree-sitter-python; Node/Express) behind the same `AnalysisModel`
TF-IDF retrieval instead of neural embeddings	no external dependency, works offline	replace `VectorIndex` with embeddings + pgvector; same interface
No Redis queue, in-process thread pool	single-server deployment	move `ProjectPipelineService` jobs to Redis/RabbitMQ workers
Call resolution is **best effort** (no type inference for lambdas/reflection)	no compiler	use JavaParser's symbol solver or compile-time bytecode analysis
Tokens in `localStorage`	simplicity	httpOnly cookie + refresh rotation
H2 default DB	zero-setup	PostgreSQL profile already provided
LLM answers untested without a key in our environment	no key available	verify with a key; add streaming responses
Large repos analysed in one pass	memory	incremental analysis, per-file caching

11. Appendix

11.1 Run commands

```
# Windows: double-click start-dev.cmd      (starts both servers)

cd backend  && mvn spring-boot:run          # API  http://localhost:8080/api
cd frontend && npm install && npm run dev   # UI   http://localhost:5173
cd backend  && mvn test                     # 10 tests

# optional LLM
set LLM_PROVIDER=anthropic   &  set LLM_API_KEY=...        (anthropic | openai | gemini)
# change the JWT secret before deploying
set JWT_SECRET=<long random string>
```

11.2 REST API summary (all under `/api`, JSON envelope `{success, message, data}`)

Method	Path	Purpose	Part
POST	`/auth/register`, `/auth/login`	create account / sign in	A
GET	`/auth/me`	current user (+ storage)	A
POST	`/projects/github`	import GitHub repo (202)	A
POST	`/projects/upload`	import ZIP (multipart, 202)	A
GET	`/projects`, `/projects/{id}`, `/projects/{id}/status`	list / details / progress	A
DELETE	`/projects/{id}`	delete project + files + chat	A
POST	`/projects/{id}/reanalyze`, `/sensitive-access`	re-run, grant `.env` access	A
GET	`/projects/{id}/overview`	stats, modules, libraries, security	B
GET	`/projects/{id}/files`, `/file?path=`, `/semantic?path=`	tree, read-only content, semantic tokens	B (data) / C (UI)
GET	`/projects/{id}/explain?kind=&id=` or `?ref=` or `?file=&line=&word=`	popup explanation	B
GET	`/projects/{id}/graph`, `/classes`, `/dependencies`, `/endpoints`, `/schema`	diagrams data	B
GET	`/projects/{id}/flow?endpoint=` (or `security`)	data-flow steps	B
GET	`/projects/{id}/symbols?q=`	quick search	C
POST	`/projects/{id}/chat`	ask a question	C
GET / DELETE	`/projects/{id}/chat/history`	saved conversation / clear	C
GET	`/projects/{id}/chat/suggestions`	starter questions	C

11.3 Glossary

Term	Meaning
AST	Abstract Syntax Tree — the code as a tree of declarations/statements/expressions
Knowledge graph	nodes (classes, tables, endpoints…) + typed edges (injects, extends, maps-to…)
Chunk	a meaningful piece of code/text (method, class, layer…) that gets indexed
TF-IDF	term frequency × inverse document frequency — weights words by how specific they are
Cosine similarity	angle between two vectors; 1 = same direction (very similar)
JWT	JSON Web Token — signed token proving who the user is
Zip-slip	attack where an archive entry's path escapes the target folder (`../`)
DTO	Data Transfer Object — plain object used in API requests/responses
Layer	role of a class: controller, service, repository, entity…
Barycentre sweep	layout heuristic: place a node at the average position of its neighbours to reduce crossings

11.4 Where to find things fast (for live questions)

Question	Open this file
"Show me the security config"	`security/config/SecurityConfig.java`
"Where is the ownership check?"	`project/service/ProjectAccess.java`
"How do you extract a ZIP safely?"	`project/service/ProjectPipelineService.java` → `extractZip`
"Where are layers detected?"	`analysis/java/JavaAnalyzer.java` → `detectLayer`
"Where are method calls resolved?"	`JavaAnalyzer.resolveCalls` / `typeOf`
"Where does the ER diagram data come from?"	`analysis/SchemaAnalyzer.java`
"Where is the data flow built?"	`flow/FlowService.java` → `endpointFlow`, `walk`
"Where is vector search?"	`search/VectorIndex.java`
"Where does chat decide what to do?"	`chat/ChatEngine.java` → `route`
"Where is the architecture diagram built?"	`frontend/src/views/ArchitectureTab.jsx` → `buildOverview`
"Where is PNG export?"	`frontend/src/views/diagram/shared.jsx` → `DiagramTools`
"Where are code colours applied?"	`frontend/src/views/CodeViewer.jsx`